

Resolution in FOL

Outline

I. Logic programming

II. Resolution

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
- Symbolic manipulation (e.g., writing compilers, parsing natural languages)

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
-
- ◆ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
- ◆ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).
- // *American*(x) $\wedge$ *Weapon*(y) $\wedge$ *Hostile*(z) $\wedge$ *Sells*(x, y, z) $\Rightarrow$ *Criminal*(x)

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
- ♦ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).
- // American(x) $\wedge$ Weapon(y) $\wedge$ Hostile(z) $\wedge$ Sells(x, y, z) $\Rightarrow$ Criminal(x)*
- `criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).`

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
- ♦ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).

// American(x) $\wedge$ Weapon(y) $\wedge$ Hostile(z) $\wedge$ Sells(x, y, z) $\Rightarrow$ Criminal(x)

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).



I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
- ♦ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).

// American(x) $\wedge$ Weapon(y) $\wedge$ Hostile(z) $\wedge$ Sells(x, y, z) $\Rightarrow$ Criminal(x)

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

⊃

$\wedge$

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
 - Symbolic manipulation (e.g., writing compilers, parsing natural languages)
- ♦ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).

// American(x) $\wedge$ Weapon(y) $\wedge$ Hostile(z) $\wedge$ Sells(x,y,z) $\Rightarrow$ Criminal(x)

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

⊃

$\wedge$

uppercase letters
for variables

I. Logic Programming

Algorithm = Logic + Control (Robert Kowalski)

Prolog (1972) is the most widely used logic programming language.

- Rapid prototyping
- Symbolic manipulation (e.g., writing compilers, parsing natural languages)

- ◆ A Prolog program is a set of definite clauses (disjunction of one positive literal and ≥ 0 negative literals).

// $American(x) \wedge Weapon(y) \wedge Hostile(z) \wedge Sells(x, y, z) \Rightarrow Criminal(x)$

criminal(X) :- american(X), weapon(Y), sells(X,Y,Z), hostile(Z).

Diagram illustrating the components of a Prolog clause:

- $\Leftarrow$ (Implies)
- $\wedge$ (And)
- uppercase letters for variables
- end of a clause

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Example List appending.

```
append([], Y, Y).
```

```
append( [A|X], Y, [A|Z]) :- append(X,Y,Z).
```

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Example List appending.

```
// appending the empty list and the list Y produces the same list Y.  
append([], Y, Y).
```

```
append( [A|X], Y, [A|Z]) :- append(X,Y,Z).
```

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Example List appending.

```
// appending the empty list and the list Y produces the same list Y.  
append([], Y, Y).
```

```
append([A|X], Y, [A|Z]) :- append(X,Y,Z).
```

|
a list whose first element
is A and rest is X.

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Example List appending.

```
// appending the empty list and the list Y produces the same list Y.  
append([], Y, Y).
```

```
// [A | Z] is the result of appending [A | X] and Y provided that Z is  
// the result of appending X and Y.
```

```
append([A|X], Y, [A|Z]) :- append(X,Y,Z).
```

|
a list whose first element
is A and rest is X.

Backward Chaining in Prolog

- ◆ Prolog recursively defines a function.

Example List appending.

```
// appending the empty list and the list Y produces the same list Y.  
append([], Y, Y).
```

```
// [A | Z] is the result of appending [A | X] and Y provided that Z is  
// the result of appending X and Y.
```

```
append([A|X], Y, [A|Z]) :- append(X,Y,Z).
```

|
a list whose first element
is A and rest is X.

Describes the relations among the three arguments of append.

Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

`append(X, Y, [1, 2, 3])`

Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

`append(X, Y, [1, 2, 3])`

(1)

`append([], Y, Y)`

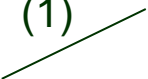
Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

`append(X, Y, [1, 2, 3])`

(1) 

`append([], Y, Y)`

`X = []`

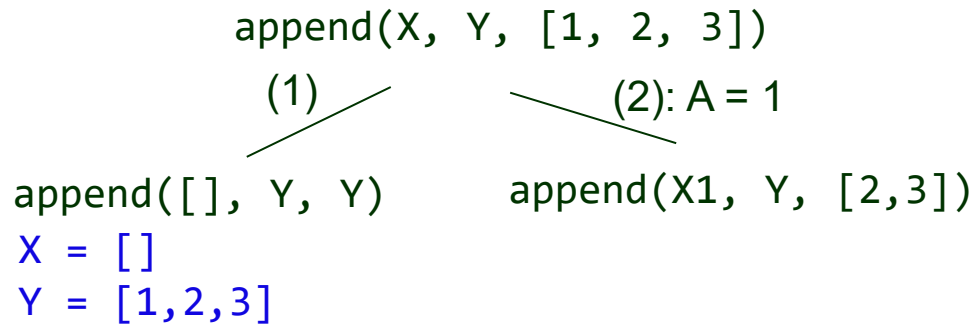
`Y = [1,2,3]`

Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

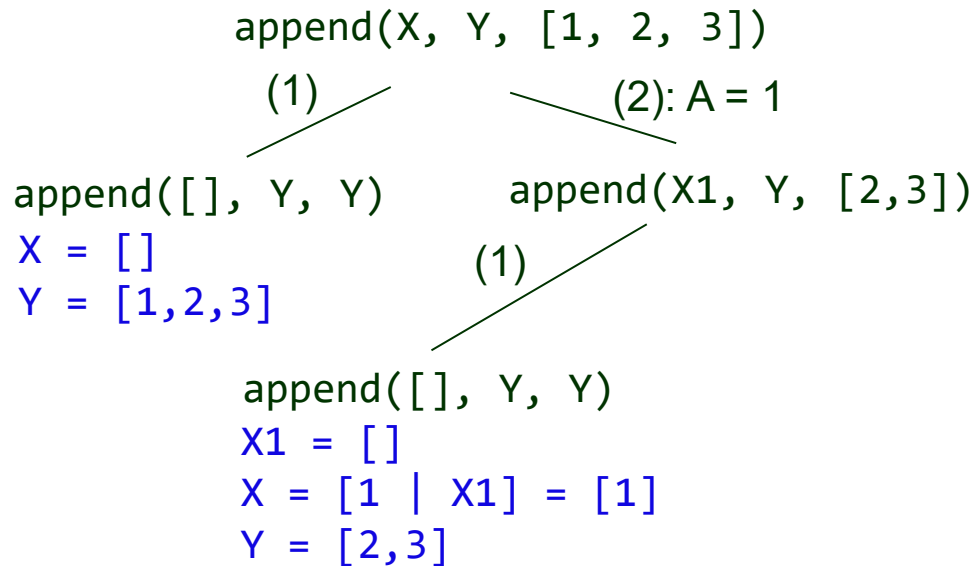


Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

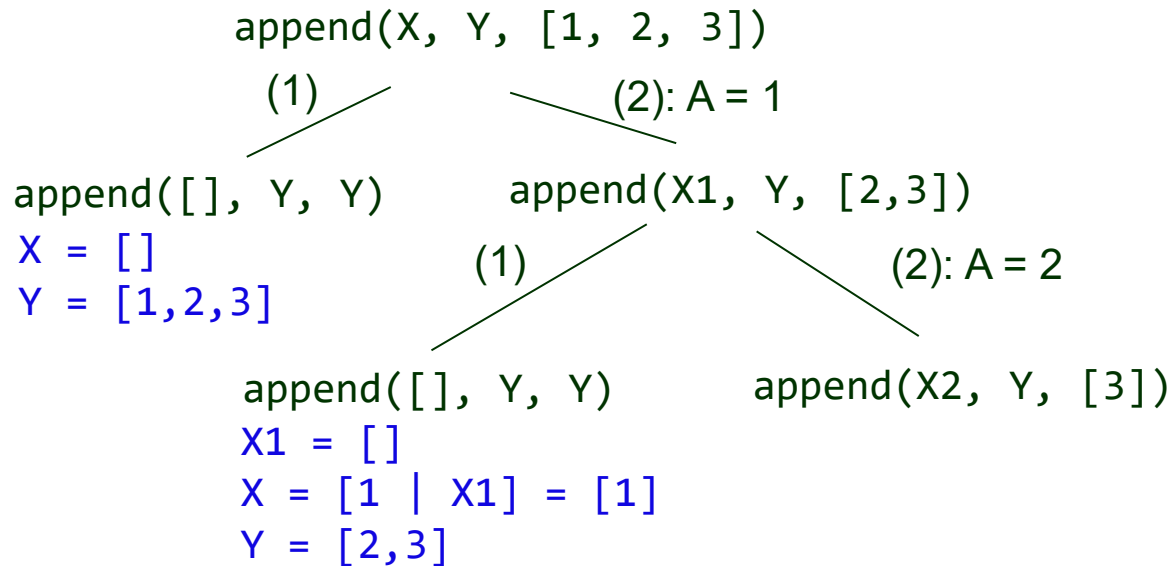


Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

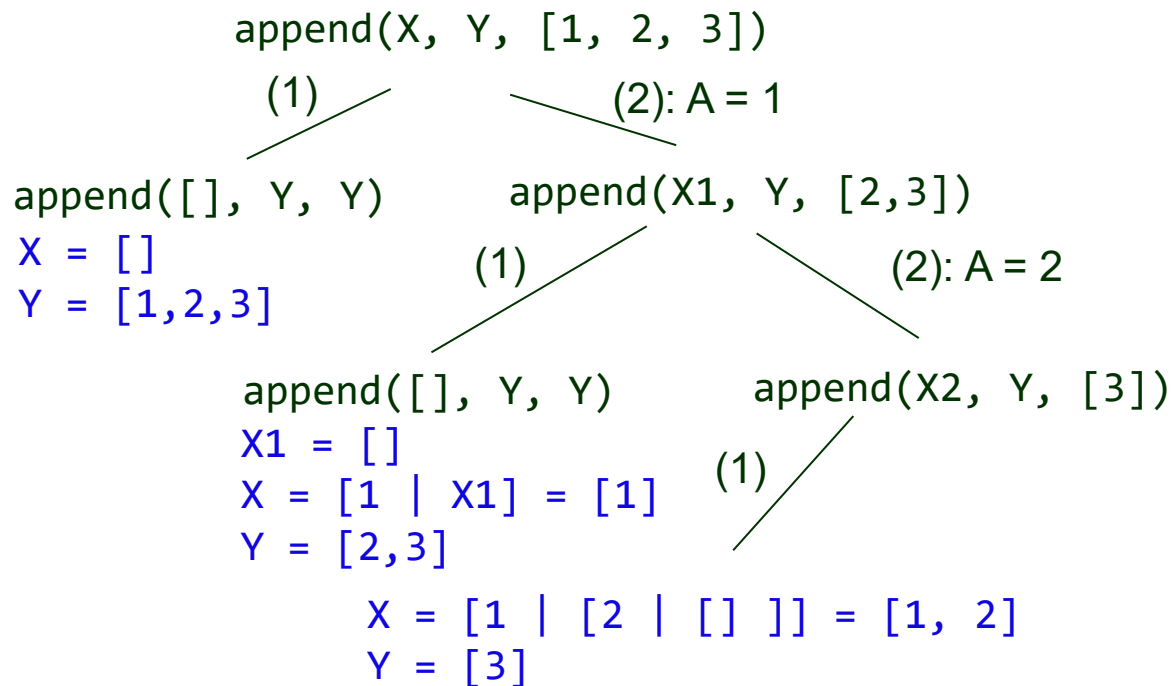


Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

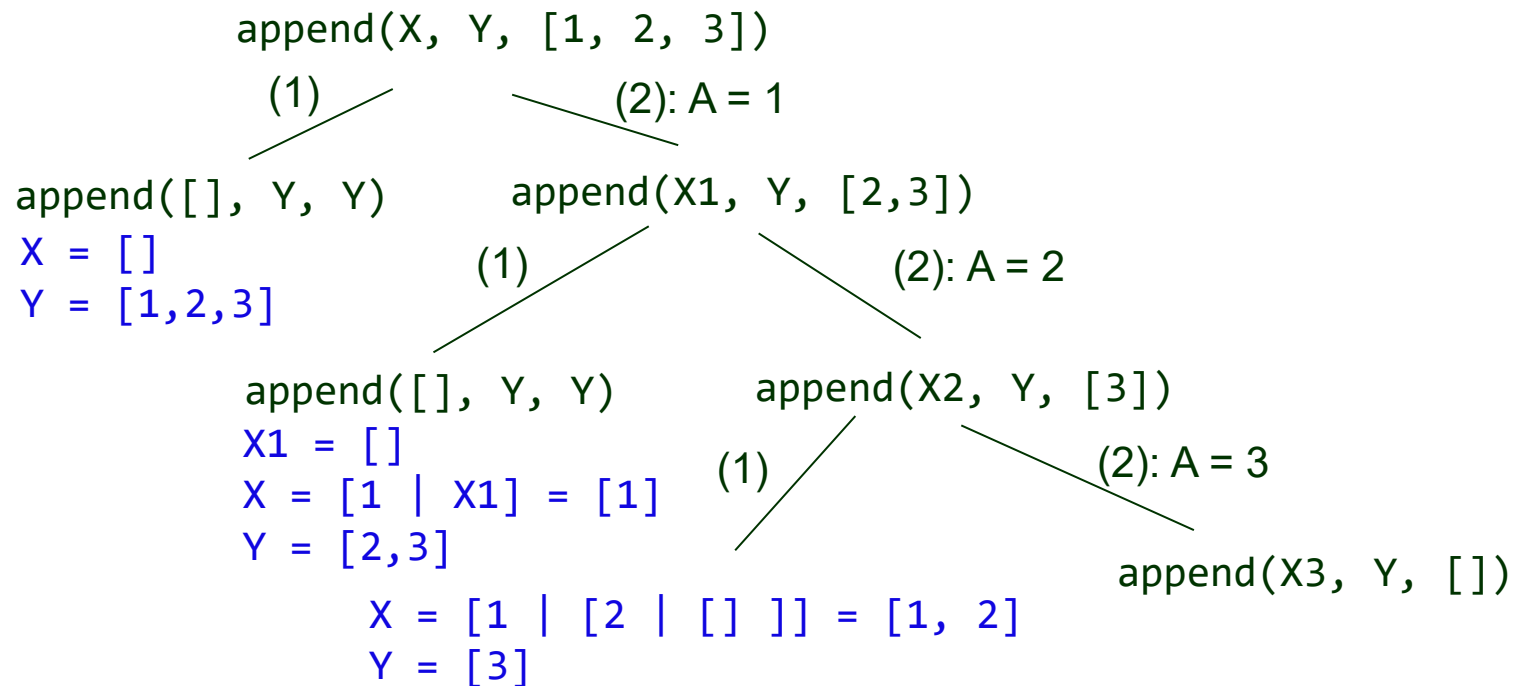


Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`

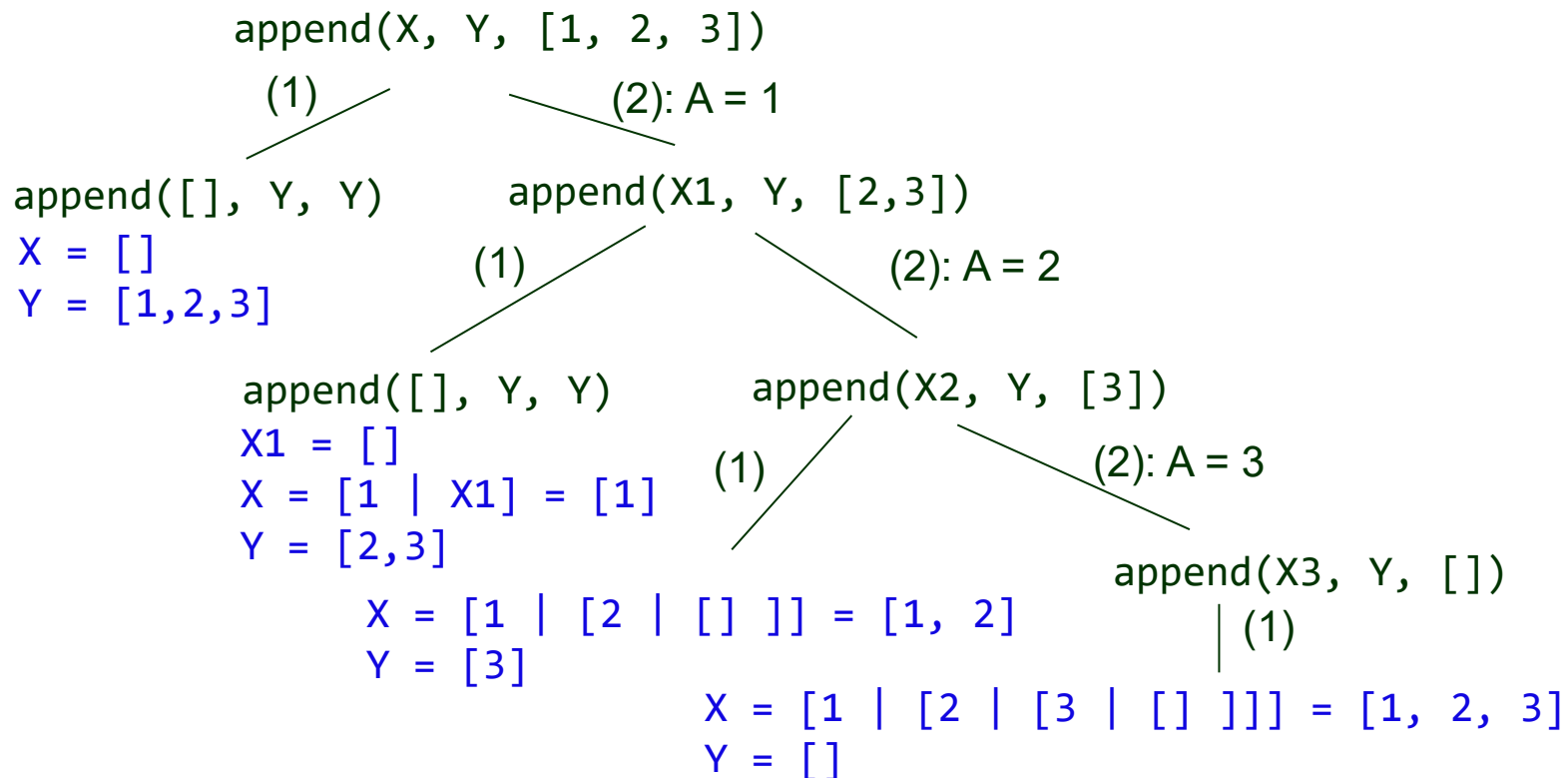


Query Example

(1) `append([], Y, Y).`

(2) `append( [A|X], Y, [A|Z]) :- append(X,Y,Z).`

Query: `append(X, Y, [1, 2, 3])`



Infinite Loop

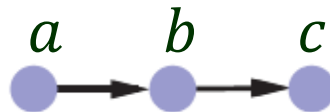
Find if a path exists between two nodes in a directed graph.

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```

Infinite Loop

Find if a path exists between two nodes in a directed graph.

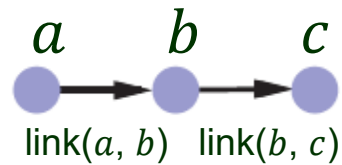
```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```



Infinite Loop

Find if a path exists between two nodes in a directed graph.

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```

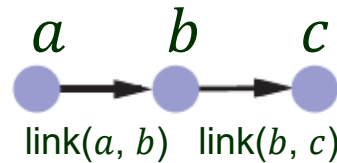


Infinite Loop

Find if a path exists between two nodes in a directed graph.

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```

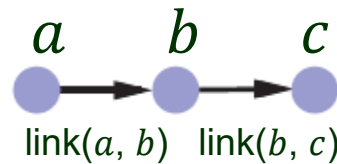
Query path(a, c)



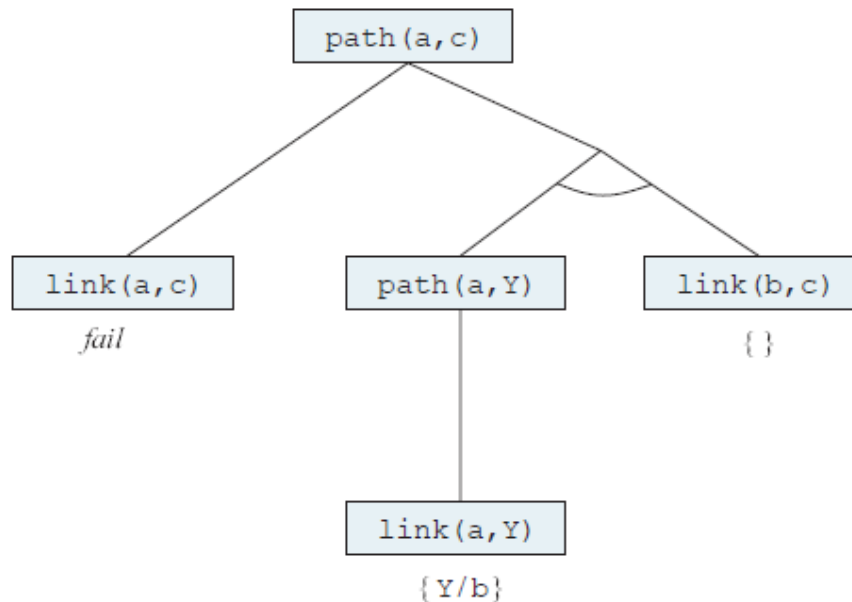
Infinite Loop

Find if a path exists between two nodes in a directed graph.

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```



Query $\text{path}(a, c)$



Infinite Loop

Find if a path exists between two nodes in a directed graph.

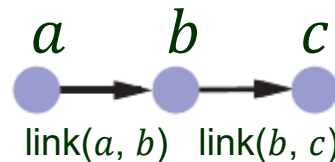
```
path(X,Z) :- link(X, Z).
```

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

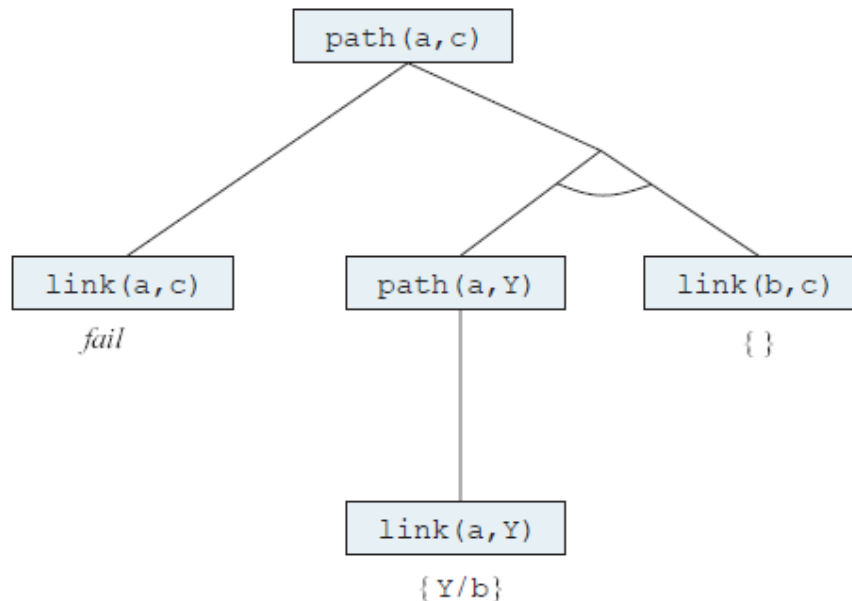
switch order

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

```
path(X,Z) :- link(X, Z).
```



Query path(a, c)



Infinite Loop

Find if a path exists between two nodes in a directed graph.

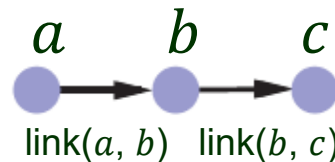
```
path(X,Z) :- link(X, Z).
```

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

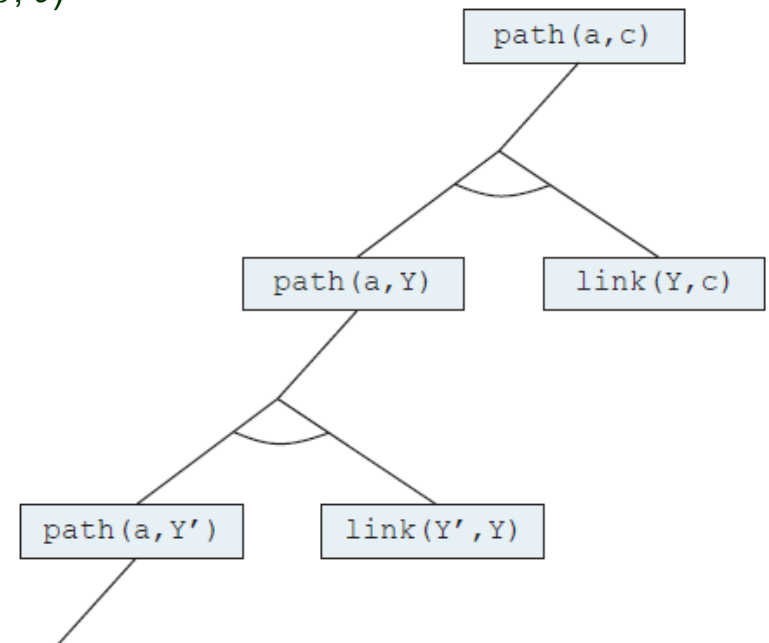
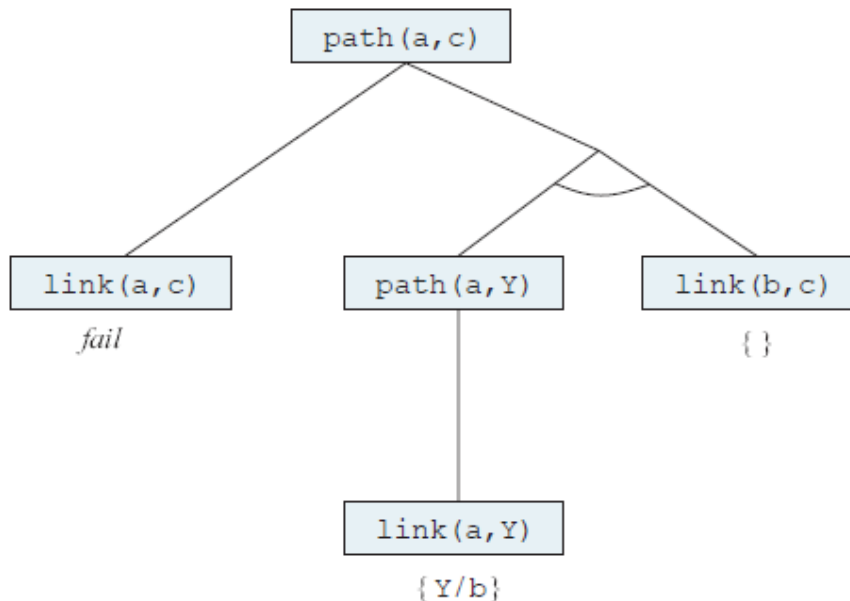
switch order

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

```
path(X,Z) :- link(X, Z).
```



Query $\text{path}(a, c)$



Infinite Loop

Find if a path exists between two nodes in a directed graph.

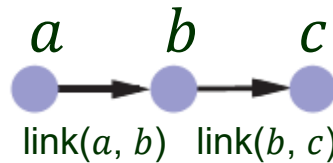
```
path(X,Z) :- link(X, Z).
```

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

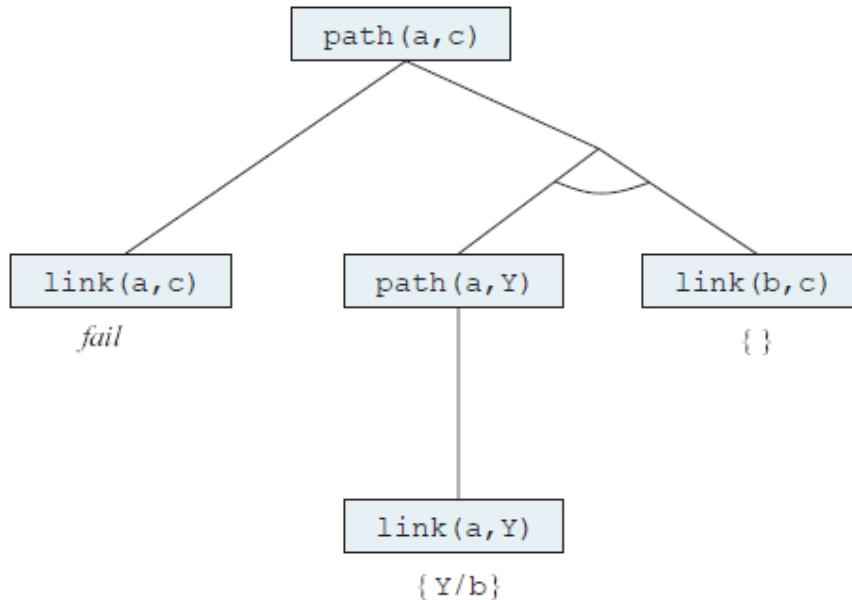
switch order

```
path(X,Z) :- path(X,Y), link(Y,Z).
```

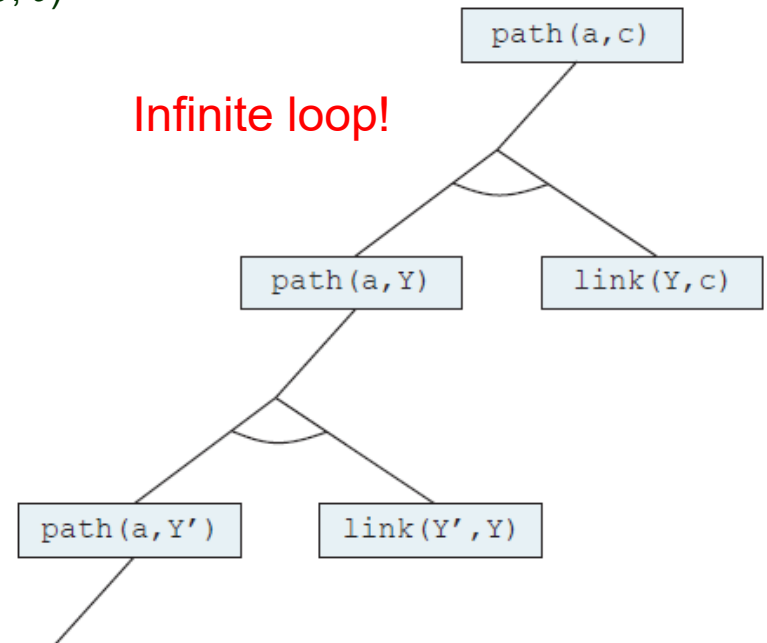
```
path(X,Z) :- link(X, Z).
```



Query $\text{path}(a, c)$

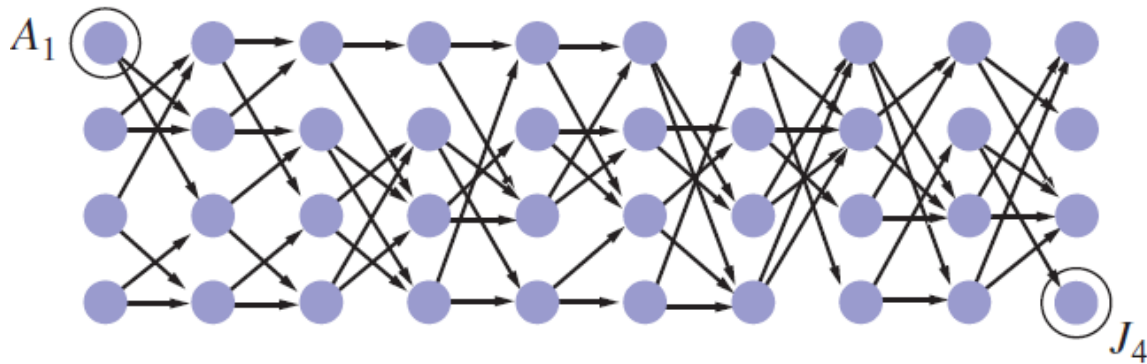


Infinite loop!



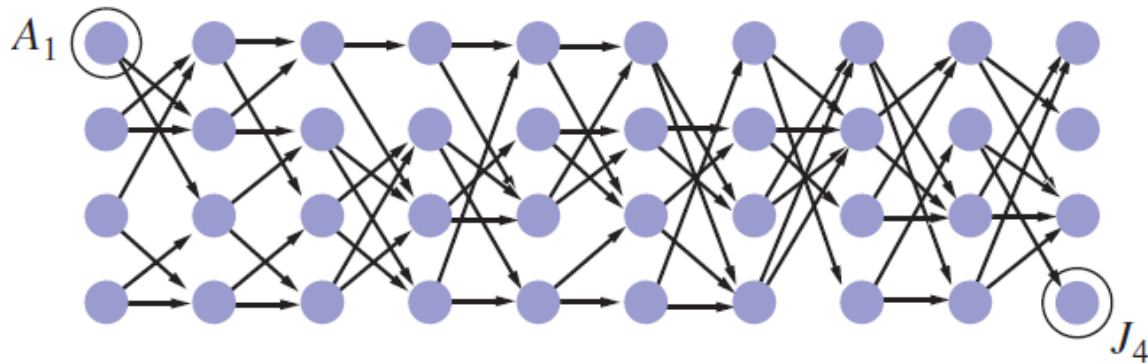
Redundant Inference

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```



Redundant Inference

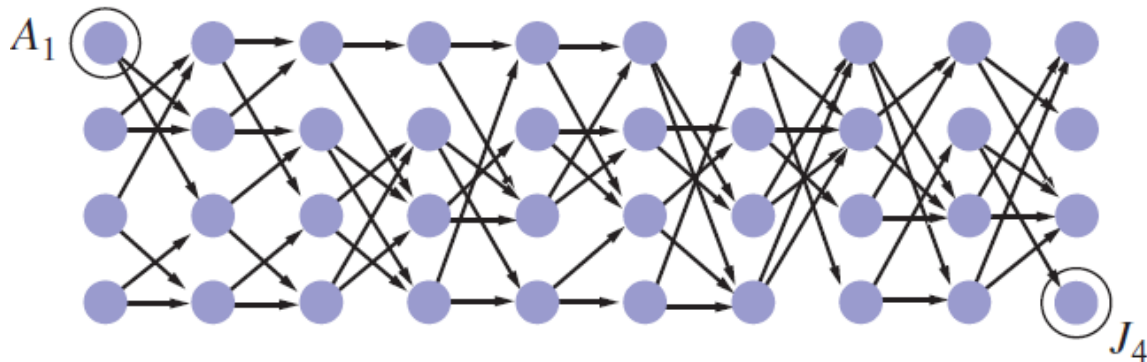
```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```



Query $\text{path}(A_1, J_4)$

Redundant Inference

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```

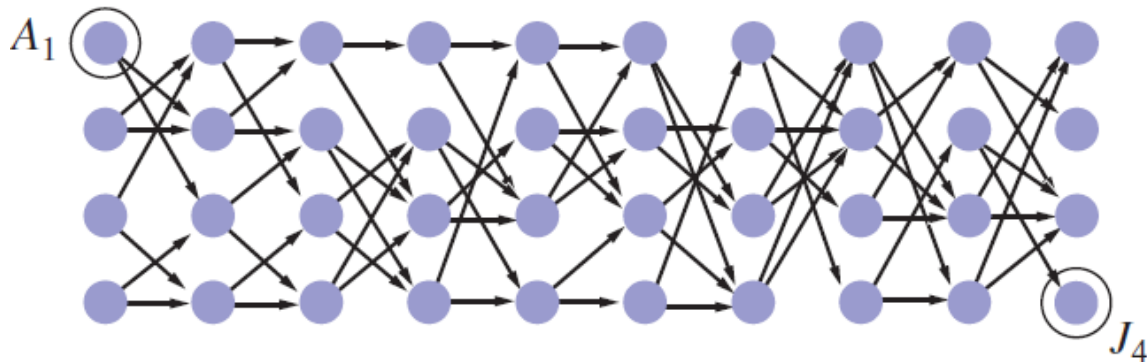


Query $\text{path}(A_1, J_4)$

- ♠ Prolog performs 877 inferences (most of which involve nodes from which the goal is unreachable).

Redundant Inference

```
path(X,Z) :- link(X, Z).  
path(X,Z) :- path(X,Y), link(Y,Z).
```



Query path(A1, J4)

- ♠ Prolog performs 877 inferences (most of which involve nodes from which the goal is unreachable).
- ♦ Forward chaining performs only 62 inferences.

II. Resolution Inference Rule

$$l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_k$$

$$\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)$$

where $\theta = \text{UNIFY}(l_i, m_j)$ makes l_i and m_j two complementary literals.

II. Resolution Inference Rule

$$l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_k$$

$$\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)$$

where $\theta = \text{UNIFY}(l_i, m_j)$ makes l_i and m_j two complementary literals.

$$\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$$

$$(\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v))$$

II. Resolution Inference Rule

$$l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_k$$

$$\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)$$

where $\theta = \text{UNIFY}(l_i, m_j)$ makes l_i and m_j two complementary literals.

$$\underbrace{\text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \quad (\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v))}_{\text{unifier: } \theta = \{u/G(x), v/x\}}$$

II. Resolution Inference Rule

$$l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_k$$

$$\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)$$

where $\theta = \text{UNIFY}(l_i, m_j)$ makes l_i and m_j two complementary literals.

$$\begin{array}{c} \text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \quad (\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v)) \\ \underbrace{\hspace{15em}} \\ \text{unifier: } \theta = \{u/G(x), v/x\} \end{array}$$

$$\text{resolvent: } \text{Animal}(F(x)) \vee \neg \text{Kills}(G(x), x)$$

II. Resolution Inference Rule

$$l_1 \vee \cdots \vee l_i \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_j \vee \cdots \vee m_k$$

$$\text{SUBST}(\theta, l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)$$

where $\theta = \text{UNIFY}(l_i, m_j)$ makes l_i and m_j two complementary literals.

$$\begin{array}{c} \text{Animal}(F(x)) \vee \text{Loves}(G(x), x) \quad (\neg \text{Loves}(u, v) \vee \neg \text{Kills}(u, v)) \\ \underbrace{\hspace{15em}} \\ \text{unifier: } \theta = \{u/G(x), v/x\} \end{array}$$

$$\text{resolvent: } \text{Animal}(F(x)) \vee \neg \text{Kills}(G(x), x)$$

- ♣ Binary resolution as given above does not yield a complete inference procedure.
- ♣ **Full resolution** does. It resolves subsets of literals in each clause that are unifiable.

Example Proof 1

The crime example:

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$

$\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$

$\neg \text{Missile}(x) \vee \text{Weapon}(x)$

$\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$

$\text{Owns}(\text{Nono}, M_1)$

$\text{Missile}(M_1)$

$\text{American}(\text{West})$

$\text{Enemy}(\text{Nono}, \text{America})$

Example Proof 1

The crime example:

$\neg \text{American}(x) \vee \neg \text{Weapon}(y) \vee \neg \text{Sells}(x, y, z) \vee \neg \text{Hostile}(z) \vee \text{Criminal}(x)$

$\neg \text{Missile}(x) \vee \neg \text{Owns}(\text{Nono}, x) \vee \text{Sells}(\text{West}, x, \text{Nono})$

$\neg \text{Missile}(x) \vee \text{Weapon}(x)$

$\neg \text{Enemy}(x, \text{America}) \vee \text{Hostile}(x)$

$\text{Owns}(\text{Nono}, M_1)$

$\text{Missile}(M_1)$

$\text{American}(\text{West})$

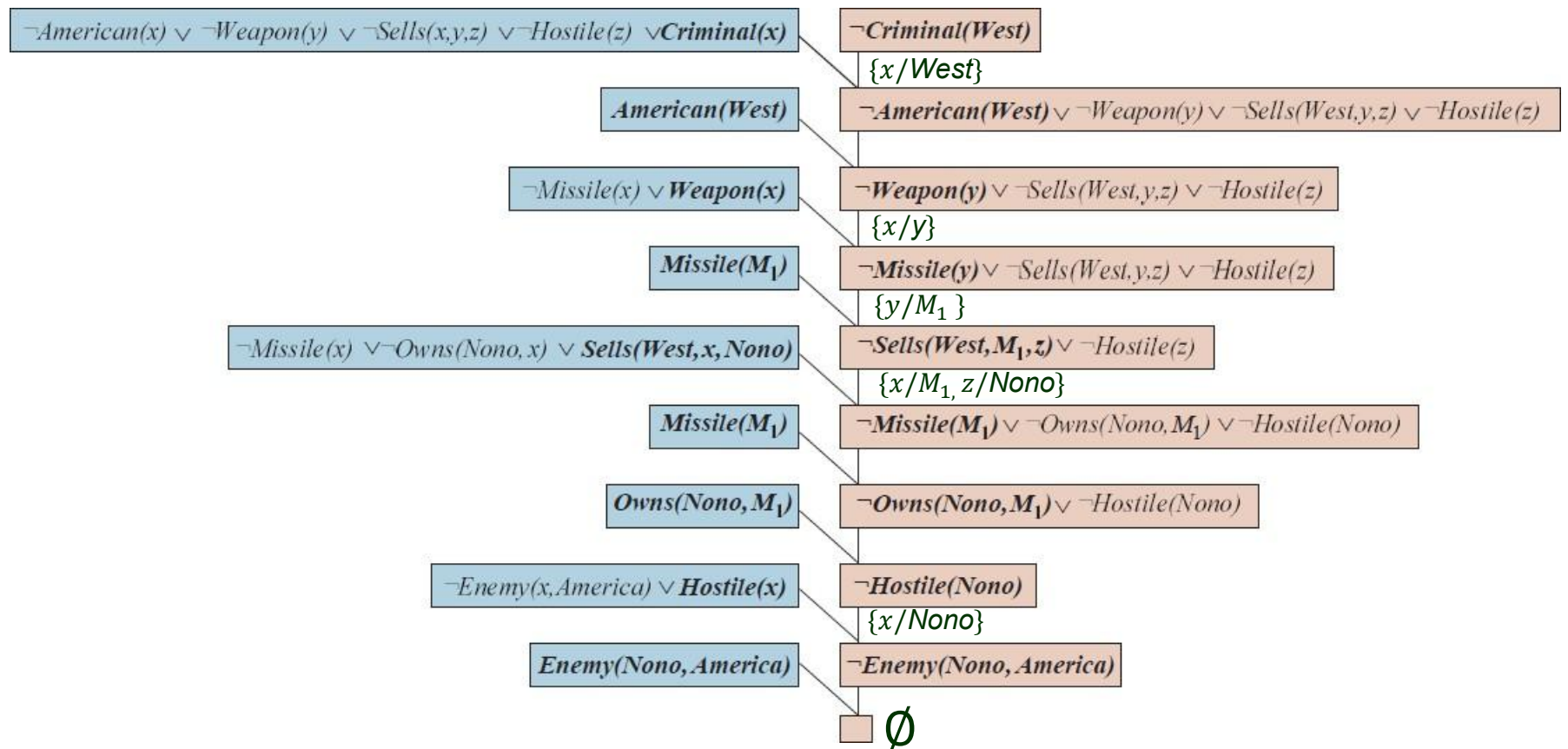
$\text{Enemy}(\text{Nono}, \text{America})$

We prove $\text{Criminal}(\text{West})$ by adding

$\neg \text{Criminal}(\text{West})$

and deriving the empty clause $\emptyset$.

Resolution Proof 1



- ♦ Is goal-driven like backward chaining on the main spine but progresses in a forward manner instead of recursively.
- ♦ Always resolve with a clause containing a positive literal that unifies with the leftmost literal of the “current” clause on the spine.

Example Proof 2 (with Skolemization)

Everyone who loves all animals is loved by someone.

Anyone who kills an animal is loved by no one.

Jack loves all animals.

Either Jack or Curiosity killed the cat, who is named Tuna.

Did Curiosity kill the cat?

Example Proof 2 (with Skolemization)

*Everyone who loves all animals is loved by someone.
Anyone who kills an animal is loved by no one.
Jack loves all animals.
Either Jack or Curiosity killed the cat, who is named Tuna.
Did Curiosity kill the cat?*

A. $\forall x \left(\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y) \right) \Rightarrow \exists y \text{ Loves}(y, x)$

Example Proof 2 (with Skolemization)

*Everyone who loves all animals is loved by someone.
Anyone who kills an animal is loved by no one.
Jack loves all animals.
Either Jack or Curiosity killed the cat, who is named Tuna.
Did Curiosity kill the cat?*

A. $\forall x \left(\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y) \right) \Rightarrow \exists y \text{ Loves}(y, x)$

B. $\forall x \left(\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z) \right) \Rightarrow (\forall y \neg \text{Loves}(y, x))$

Example Proof 2 (with Skolemization)

Everyone who loves all animals is loved by someone.

Anyone who kills an animal is loved by no one.

Jack loves all animals.

Either Jack or Curiosity killed the cat, who is named Tuna.

Did Curiosity kill the cat?

A. $\forall x \left(\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y) \right) \Rightarrow \exists y \text{ Loves}(y, x)$

B. $\forall x \left(\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z) \right) \Rightarrow (\forall y \neg \text{Loves}(y, x))$

C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$

Example Proof 2 (with Skolemization)

Everyone who loves all animals is loved by someone.

Anyone who kills an animal is loved by no one.

Jack loves all animals.

Either Jack or Curiosity killed the cat, who is named Tuna.

Did Curiosity kill the cat?

A. $\forall x \left(\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y) \right) \Rightarrow \exists y \text{ Loves}(y, x)$

B. $\forall x \left(\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z) \right) \Rightarrow (\forall y \neg \text{Loves}(y, x))$

C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$

D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$

Example Proof 2 (with Skolemization)

Everyone who loves all animals is loved by someone.
Anyone who kills an animal is loved by no one.
Jack loves all animals.
Either Jack or Curiosity killed the cat, who is named Tuna.
Did Curiosity kill the cat?

A. $\forall x \left(\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y) \right) \Rightarrow \exists y \text{ Loves}(y, x)$

B. $\forall x \left(\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z) \right) \Rightarrow (\forall y \neg \text{Loves}(y, x))$

C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$

D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$

E. $\text{Cat}(\text{Tuna})$

F. $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$

Background
knowledge {

Example Proof 2 (with Skolemization)

Everyone who loves all animals is loved by someone.
Anyone who kills an animal is loved by no one.
Jack loves all animals.
Either Jack or Curiosity killed the cat, who is named Tuna.
Did Curiosity kill the cat?

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow \exists y \text{ Loves}(y, x)$

B. $\forall x (\exists z \text{ Animal}(z) \wedge \text{Kills}(x, z)) \Rightarrow (\forall y \neg \text{Loves}(y, x))$

C. $\forall x \text{ Animal}(x) \Rightarrow \text{Loves}(\text{Jack}, x)$

D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$

E. $\text{Cat}(\text{Tuna})$

F. $\forall x \text{ Cat}(x) \Rightarrow \text{Animal}(x)$

$\neg$ G. $\neg \text{Kills}(\text{Curiosity}, \text{Tuna})$

Background
knowledge {

Negated
goal

Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$

Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$
 $\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$



Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$

$$\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$

$$\forall x (\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$



Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$

$$\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$

$$\forall x (\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$

$$(\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))) \vee \text{Loves}(G(x), x)$$



Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$

$$\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$

$$\forall x (\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$$

$$(\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))) \vee \text{Loves}(G(x), x)$$



A1. $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$

A2. $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$



Converting to CNF

A. $\forall x (\forall y \text{ Animal}(y) \Rightarrow \text{Loves}(x, y)) \Rightarrow (\exists y \text{ Loves}(y, x))$

$\forall x \neg(\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$

$\forall x (\exists y \text{ Animal}(y) \wedge \neg \text{Loves}(x, y)) \vee (\exists y \text{ Loves}(y, x))$

$(\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))) \vee \text{Loves}(G(x), x)$



A1. $\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)$

A2. $\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)$

B. $\neg \text{Animal}(z) \vee \neg \text{Kills}(x, z) \vee \neg \text{Loves}(y, x)$

C. $\neg \text{Animal}(x) \vee \text{Loves}(\text{Jack}, x)$

D. $\text{Kills}(\text{Jack}, \text{Tuna}) \vee \text{Kills}(\text{Curiosity}, \text{Tuna})$

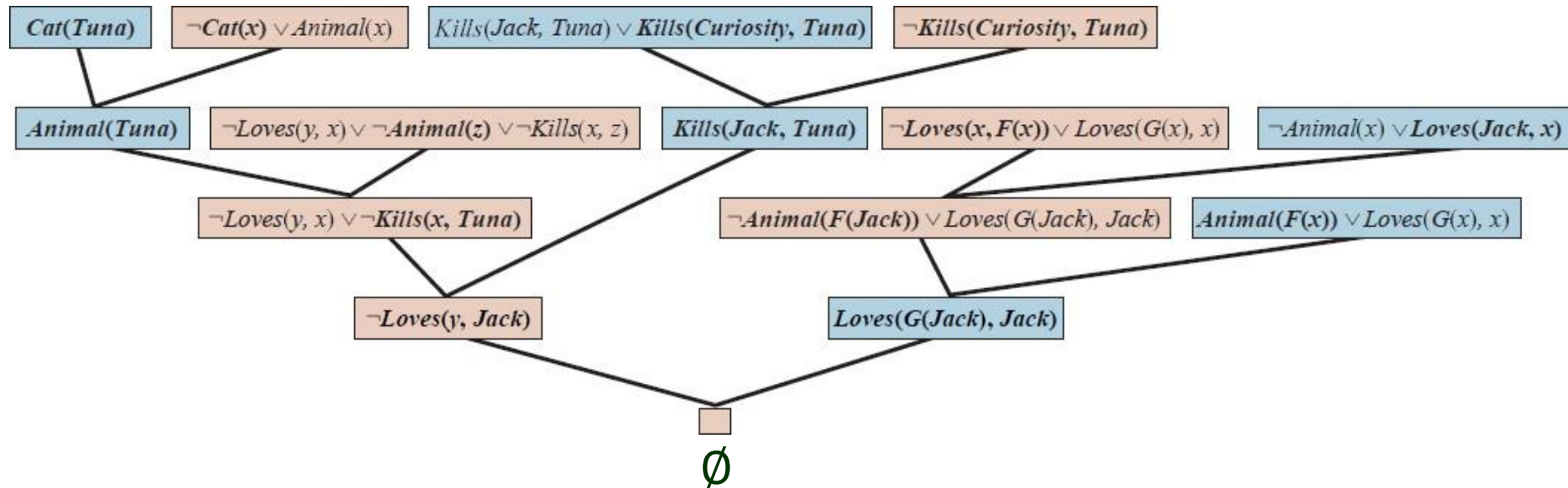
E. $\text{Cat}(\text{Tuna})$

F. $\neg \text{Cat}(x) \vee \text{Animal}(x)$

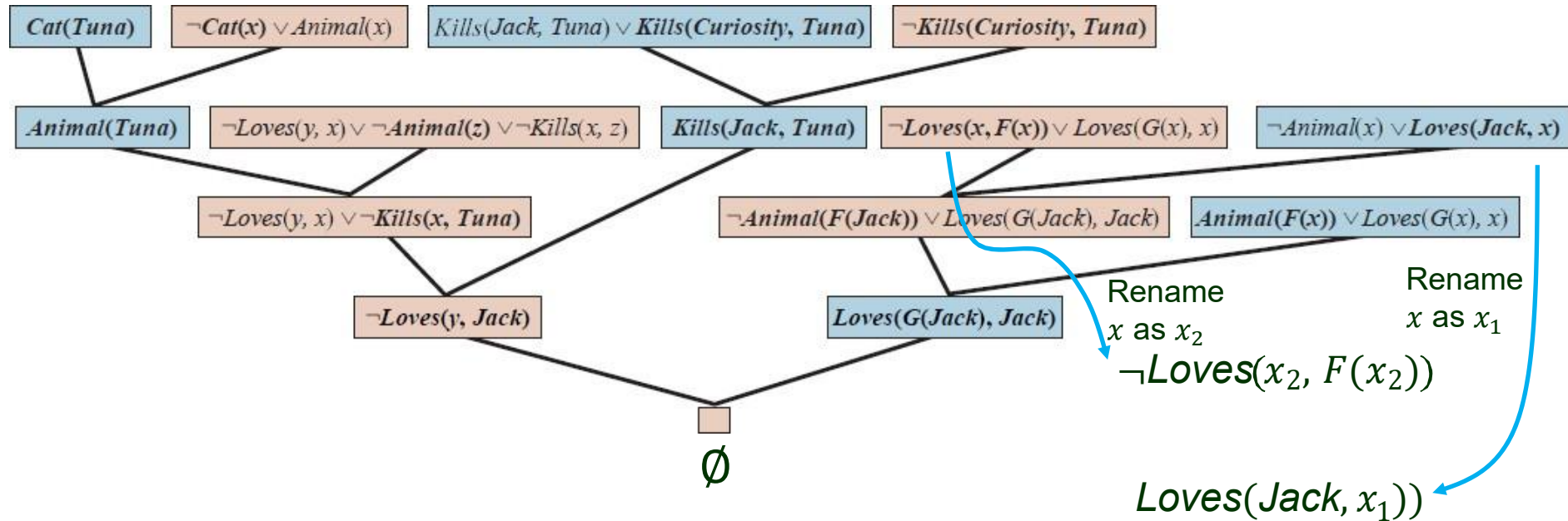
$\neg$ G. $\neg \text{Kills}(\text{Curiosity}, \text{Tuna})$



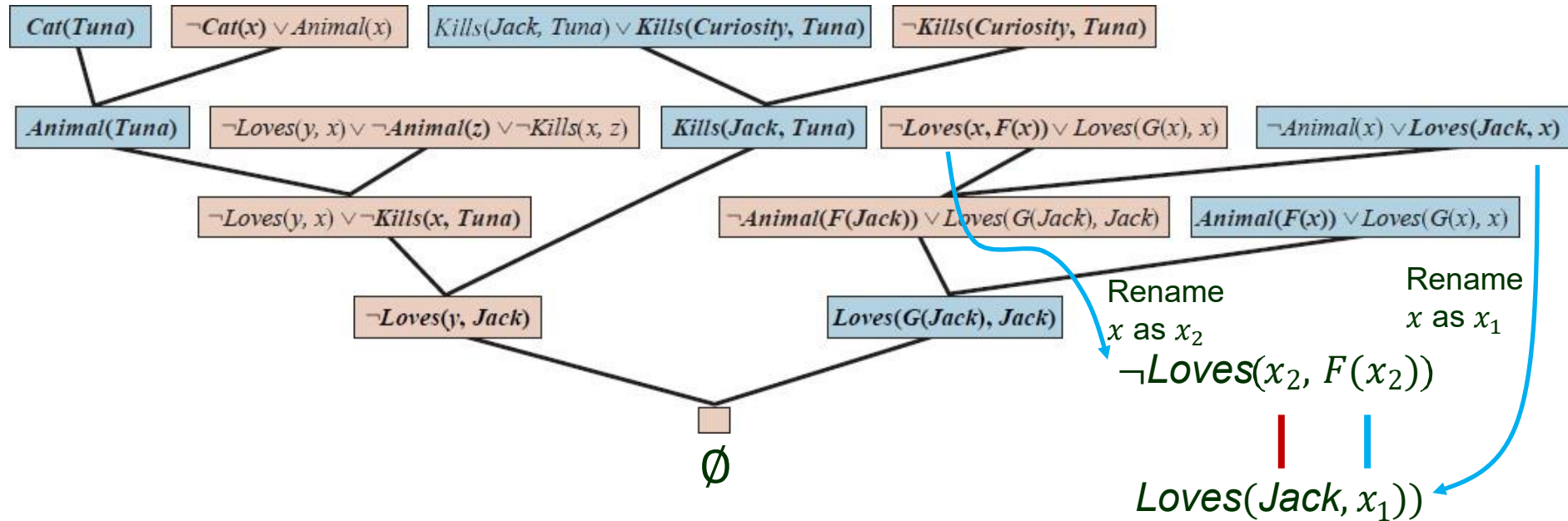
Resolution Proof 2



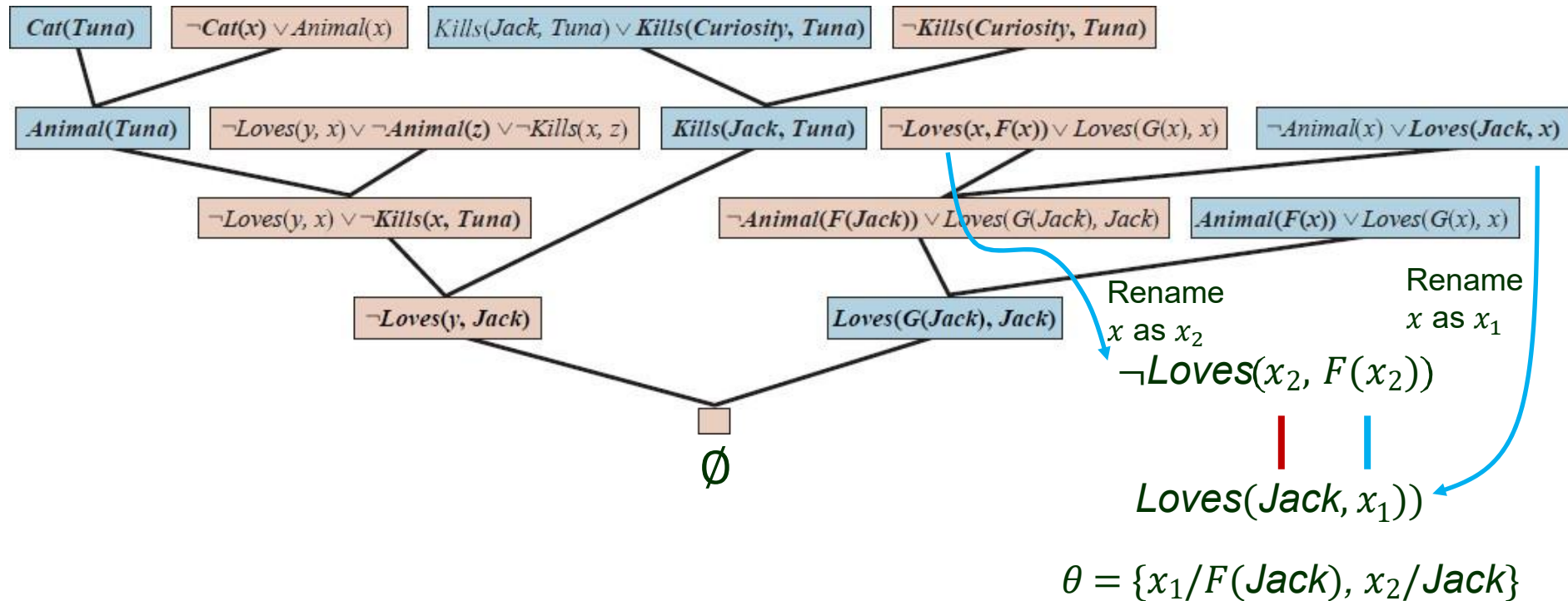
Resolution Proof 2

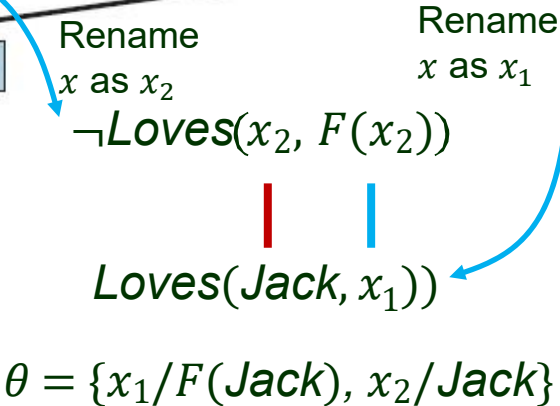


Resolution Proof 2

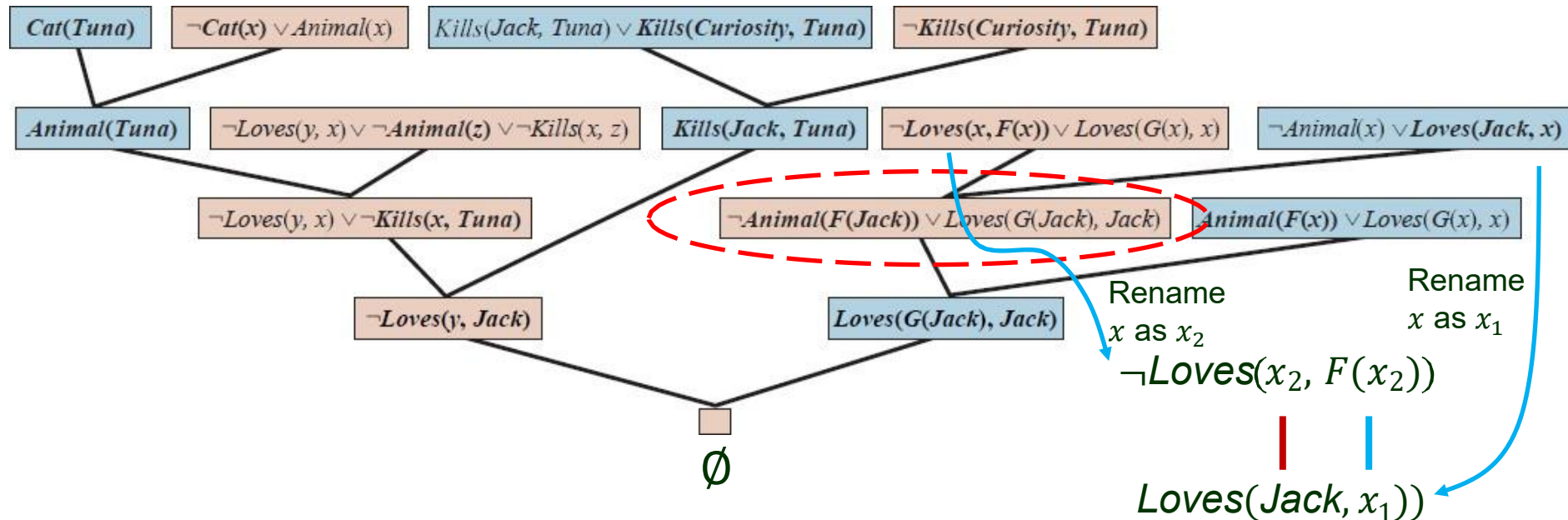


Resolution Proof 2





Resolution Proof 2



Paraphrased in English:

$$\theta = \{x_1/F(Jack), x_2/Jack\}$$

Suppose Curiosity did not kill Tuna. We know that either Jack or Curiosity did; thus Jack must have. Now, Tuna is a cat and cats are animals, so Tuna is an animal. Because anyone who kills an animal is loved by no one, we know that no one loves Jack. On the other hand, Jack loves all animals, so someone loves him; so we have a contradiction. Therefore, Curiosity killed the cat.

Completeness of Resolution

Theorem If a set S of sentences is unsatisfiable, then resolution will always be able to derive a contradiction.

Not all logical consequences of S can be generated using resolution.

A sentence entailed by S can always be established using resolution.

Completeness of Resolution

Theorem If a set S of sentences is unsatisfiable, then resolution will always be able to derive a contradiction.

Not all logical consequences of S can be generated using resolution.

A sentence entailed by S can always be established using resolution.

We can use resolution to find all answers to a question $Q(x)$ by proving that $KB \wedge \neg Q(x)$ is unsatisfiable.